

Report Progetto 1

Moltiplicazione di Matrici Parallela con OpenMP

Tecniche di Programmazione Avanzata ed AI – TPA 2026

Matteo Pettinacci

Aprile 2026

Indice

1	Introduzione all'algoritmo e al problema	2
2	Descrizione del codice sequenziale	2
3	Codice parallelo: granularità di assegnazione del lavoro	2
3.1	Variante per righe: assegnazione di un gruppo di elementi	3
3.2	Variante per elemento: assegnazione di un singolo elemento per thread	3
3.3	Variante private: buffer locale per riga	3
4	Descrizione del benchmark	4
5	Dati sperimentali completi	4
5.1	N = 512 — baseline sequenziale: 27.217 ms	4
5.2	N = 1024 — baseline sequenziale: 64.570 ms	6
5.3	N = 2048 — baseline sequenziale: 2784.500 ms	8
6	Studio delle variabili condivise/private	11
6.1	Differenza architetturale	11
6.2	Risultati e discussione	11
7	Studio dello scheduling dei thread	12
7.1	I tre scheduler	12
7.2	Confronto scheduler e chunk size	12
8	Studio delle performance al variare della dimensione e del parallelismo	13
9	Conclusioni	14

1 Introduzione all'algoritmo e al problema

Questo progetto implementa e ottimizza la **moltiplicazione di matrici quadrate** ($C = A \times B$) in C++ con OpenMP. L'algoritmo è definito da:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} \cdot B_{kj}, \quad \forall i, j \in [0, n) \quad (1)$$

La complessità computazionale è $O(n^3)$, ma il problema è classificato come *embarrassingly parallel*: ogni elemento C_{ij} è calcolabile in modo completamente indipendente dagli altri, eliminando la necessità di sincronizzazione tra thread durante il calcolo.

Le matrici sono di tipo `float` a 32 bit. Le dimensioni scelte ($n = 512, 1024, 2048$) sono multipli del numero di core logici della macchina di test e potenze di 2. Lo studio delle performance è strutturato attorno a quattro variabili indipendenti: dimensione del problema, numero di thread, distribuzione del lavoro tra i thread (chunk size e granularità di assegnazione), e modalità di gestione della memoria (variabili *shared* vs *private*). Il tempo di I/O è escluso da tutte le misurazioni.

2 Descrizione del codice sequenziale

Il codice è organizzato attorno alla classe `Matrix`, che incapsula un array contiguo di `float` in formato *row-major*. La funzione sequenziale adotta l'ordinamento dei cicli `ikj`, più *cache-friendly* rispetto all'ordinamento *naive* `ijk`:

Listing 1: Versione sequenziale con ciclo `ikj`

```
1 void matrixMultSequential(const Matrix& A, const Matrix& B, Matrix& C)
2 {
3     const size_t n = A.getRows();
4     C.zeros();
5     for (size_t i = 0; i < n; ++i) {
6         for (size_t k = 0; k < n; ++k) {
7             const float aik = A.at(i, k); // scalare letto una sola
8             // volta
9             for (size_t j = 0; j < n; ++j)
10                 C.at(i, j) += aik * B.at(k, j);
11         }
12     }
13 }
```

Con `ikj`, per ogni coppia (i, k) fissata si itera su j : sia B_{kj} che C_{ij} vengono letti/scritti sequenzialmente, massimizzando il riutilizzo della cache. Il valore A_{ik} è caricato una volta sola e riutilizzato n volte.

3 Codice parallelo: granularità di assegnazione del lavoro

La parallelizzazione offre due livelli di granularità: assegnare a ciascun thread un **gruppo di righe** (parallelismo sul ciclo esterno i) oppure un **singolo elemento** C_{ij} con `collapse(2)`.

3.1 Variante per righe: assegnazione di un gruppo di elementi

Listing 2: Parallelismo per righe

```
1 #pragma omp parallel for schedule(runtime) default(none) shared(A, B, C
  , n)
2 for (size_t i = 0; i < n; ++i) {
3     for (size_t k = 0; k < n; ++k) {
4         const float aik = A.at(i, k);
5         for (size_t j = 0; j < n; ++j)
6             C.at(i, j) += aik * B.at(k, j);
7     }
8 }
```

Ogni thread riceve mediamente n/N_{thread} righe. L'accesso rimane sequenziale, preservando la località dell'ordinamento ikj .

3.2 Variante per elemento: assegnazione di un singolo elemento per thread

Listing 3: Parallelismo per elemento con collapse(2)

```
1 #pragma omp parallel for collapse(2) schedule(runtime) default(none)
  shared(A, B, C, n)
2 for (size_t i = 0; i < n; ++i) {
3     for (size_t j = 0; j < n; ++j) {
4         float sum = 0.0f;
5         for (size_t k = 0; k < n; ++k)
6             sum += A.at(i, k) * B.at(k, j);
7         C.at(i, j) = sum;
8     }
9 }
10 }
```

Espone n^2 unità di lavoro invece di n , ma l'ordinamento ij causa accessi non sequenziali a B (stride = n), degradando la località.

3.3 Variante private: buffer locale per riga

Listing 4: Variante private con buffer locale

```
1 #pragma omp parallel for schedule(runtime) default(none) shared(A, B, C
  , n)
2 for (size_t i = 0; i < n; ++i) {
3     float row_buf[4096] = {}; // PRIVATO: sullo stack del thread
4     for (size_t k = 0; k < n; ++k) {
5         const float aik = A.at(i, k);
6         for (size_t j = 0; j < n; ++j)
7             row_buf[j] += aik * B.at(k, j);
8     }
9     for (size_t j = 0; j < n; ++j)
10         C.at(i, j) = row_buf[j]; // flush su C una sola volta per
  riga
11 }
```

4 Descrizione del benchmark

I test sono eseguiti su un processore con 8 core logici. Le variabili sistematicamente testate sono:

- **Numero di thread:** 1, 2, 4, 8
- **Scheduler:** static, dynamic, guided
- **Chunk size:** 1, 8, 32, 128
- **Variante memoria:** shared e private

Le metriche calcolate sono speedup $S = T_{seq}/T_{par}$ ed efficienza $E = S/N_{thread}$. La baseline sequenziale è misurata senza OpenMP attivo.

5 Dati sperimentali completi

Le tre tabelle seguenti riportano **tutti** i dati raccolti, organizzati per dimensione del problema. La configurazione con speedup massimo è in grassetto.

5.1 N = 512 — baseline sequenziale: 27.217 ms

Tabella 1: Dati completi per N=512.

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
1	static	shared	1	21.114	1.289	1.289
1	static	private	1	22.528	1.208	1.208
1	static	shared	8	22.941	1.186	1.186
1	static	private	8	18.397	1.479	1.479
1	static	shared	32	19.686	1.383	1.383
1	static	private	32	11.765	2.313	2.313
1	static	shared	128	25.374	1.073	1.073
1	static	private	128	18.352	1.483	1.483
1	dynamic	shared	1	7.704	3.533	3.533
1	dynamic	private	1	10.715	2.540	2.540
1	dynamic	shared	8	7.404	3.676	3.676
1	dynamic	private	8	10.118	2.690	2.690
1	dynamic	shared	32	6.862	3.966	3.966
1	dynamic	private	32	8.534	3.189	3.189
1	dynamic	shared	128	8.287	3.284	3.284
1	dynamic	private	128	7.633	3.566	3.566
1	guided	shared	1	5.991	4.543	4.543
1	guided	private	1	8.827	3.083	3.083
1	guided	shared	8	7.253	3.753	3.753
1	guided	private	8	8.008	3.399	3.399
1	guided	shared	32	8.017	3.395	3.395
1	guided	private	32	7.888	3.450	3.450
1	guided	shared	128	6.051	4.498	4.498
1	guided	private	128	9.241	2.945	2.945

Continua nella pagina successiva

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
2	static	shared	1	16.241	1.676	0.838
2	static	private	1	7.954	3.422	1.711
2	static	shared	8	12.104	2.249	1.124
2	static	private	8	9.792	2.779	1.390
2	static	shared	32	10.179	2.674	1.337
2	static	private	32	13.489	2.018	1.009
2	static	shared	128	13.908	1.957	0.979
2	static	private	128	7.786	3.496	1.748
2	dynamic	shared	1	10.268	2.651	1.326
2	dynamic	private	1	5.271	5.164	2.582
2	dynamic	shared	8	4.247	6.408	3.204
2	dynamic	private	8	7.877	3.455	1.728
2	dynamic	shared	32	4.565	5.962	2.981
2	dynamic	private	32	10.591	2.570	1.285
2	dynamic	shared	128	3.531	7.708	3.854
2	dynamic	private	128	5.112	5.324	2.662
2	guided	shared	1	3.273	8.314	4.157
2	guided	private	1	4.260	6.389	3.194
2	guided	shared	8	3.165	8.598	4.299
2	guided	private	8	5.076	5.362	2.681
2	guided	shared	32	3.957	6.878	3.439
2	guided	private	32	4.431	6.142	3.071
2	guided	shared	128	3.242	8.395	4.197
2	guided	private	128	4.948	5.500	2.750
4	static	shared	1	10.273	2.649	0.662
4	static	private	1	6.389	4.260	1.065
4	static	shared	8	9.926	2.742	0.685
4	static	private	8	6.494	4.191	1.048
4	static	shared	32	6.398	4.254	1.063
4	static	private	32	8.056	3.378	0.845
4	static	shared	128	14.734	1.847	0.462
4	static	private	128	11.971	2.274	0.568
4	dynamic	shared	1	3.791	7.179	1.795
4	dynamic	private	1	2.869	9.485	2.371
4	dynamic	shared	8	2.880	9.452	2.363
4	dynamic	private	8	2.154	12.637	3.159
4	dynamic	shared	32	3.185	8.545	2.136
4	dynamic	private	32	3.164	8.602	2.151
4	dynamic	shared	128	2.526	10.777	2.694
4	dynamic	private	128	2.228	12.215	3.054
4	guided	shared	1	3.281	8.294	2.073
4	guided	private	1	2.208	12.326	3.082
4	guided	shared	8	1.942	14.015	3.504
4	guided	private	8	1.975	13.784	3.446
4	guided	shared	32	2.386	11.409	2.852
4	guided	private	32	3.133	8.688	2.172
4	guided	shared	128	3.778	7.205	1.801

Continua nella pagina successiva

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
4	guided	private	128	2.734	9.954	2.489
8	static	shared	1	6.337	4.295	0.537
8	static	private	1	11.868	2.293	0.287
8	static	shared	8	5.055	5.384	0.673
8	static	private	8	11.569	2.353	0.294
8	static	shared	32	15.333	1.775	0.222
8	static	private	32	15.435	1.763	0.220
8	static	shared	128	9.876	2.756	0.344
8	static	private	128	2.630	10.348	1.294
8	dynamic	shared	1	4.459	6.104	0.763
8	dynamic	private	1	2.112	12.889	1.611
8	dynamic	shared	8	3.334	8.164	1.021
8	dynamic	private	8	2.164	12.576	1.572
8	dynamic	shared	32	2.623	10.378	1.297
8	dynamic	private	32	2.166	12.567	1.571
8	dynamic	shared	128	2.715	10.024	1.253
8	dynamic	private	128	4.646	5.859	0.732
8	guided	shared	1	2.187	12.445	1.556
8	guided	private	1	1.861	14.628	1.829
8	guided	shared	8	1.711	15.910	1.989
8	guided	private	8	1.683	16.170	2.021
8	guided	shared	32	1.841	14.781	1.848
8	guided	private	32	1.567	17.369	2.171
8	guided	shared	128	2.825	9.636	1.205
8	guided	private	128	1.705	15.961	1.995

5.2 N = 1024 — baseline sequenziale: 64.570 ms

Tabella 2: Dati completi per N=1024.

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
1	static	shared	1	68.466	0.943	0.943
1	static	private	1	68.743	0.939	0.939
1	static	shared	8	121.207	0.533	0.533
1	static	private	8	84.245	0.766	0.766
1	static	shared	32	66.928	0.965	0.965
1	static	private	32	79.954	0.808	0.808
1	static	shared	128	62.214	1.038	1.038
1	static	private	128	72.699	0.888	0.888
1	dynamic	shared	1	67.015	0.964	0.964
1	dynamic	private	1	81.550	0.792	0.792
1	dynamic	shared	8	63.559	1.016	1.016
1	dynamic	private	8	75.234	0.858	0.858
1	dynamic	shared	32	81.885	0.789	0.789
1	dynamic	private	32	178.822	0.361	0.361
1	dynamic	shared	128	195.243	0.331	0.331

Continua nella pagina successiva

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
1	dynamic	private	128	231.669	0.279	0.279
1	guided	shared	1	318.029	0.203	0.203
1	guided	private	1	234.055	0.276	0.276
1	guided	shared	8	299.499	0.216	0.216
1	guided	private	8	258.865	0.249	0.249
1	guided	shared	32	260.237	0.248	0.248
1	guided	private	32	180.863	0.357	0.357
1	guided	shared	128	286.447	0.225	0.225
1	guided	private	128	243.659	0.265	0.265
2	static	shared	1	31.990	2.018	1.009
2	static	private	1	33.643	1.919	0.960
2	static	shared	8	33.466	1.929	0.964
2	static	private	8	37.970	1.701	0.851
2	static	shared	32	35.488	1.819	0.909
2	static	private	32	35.913	1.798	0.899
2	static	shared	128	34.751	1.858	0.929
2	static	private	128	35.403	1.824	0.912
2	dynamic	shared	1	38.535	1.676	0.838
2	dynamic	private	1	36.031	1.792	0.896
2	dynamic	shared	8	34.427	1.876	0.938
2	dynamic	private	8	35.673	1.810	0.905
2	dynamic	shared	32	116.428	0.555	0.277
2	dynamic	private	32	123.031	0.525	0.262
2	dynamic	shared	128	71.128	0.908	0.454
2	dynamic	private	128	164.166	0.393	0.197
2	guided	shared	1	151.427	0.426	0.213
2	guided	private	1	138.335	0.467	0.233
2	guided	shared	8	209.594	0.308	0.154
2	guided	private	8	131.666	0.490	0.245
2	guided	shared	32	72.720	0.888	0.444
2	guided	private	32	137.167	0.471	0.235
2	guided	shared	128	65.412	0.987	0.493
2	guided	private	128	171.486	0.377	0.188
4	static	shared	1	18.042	3.579	0.895
4	static	private	1	18.813	3.432	0.858
4	static	shared	8	21.837	2.957	0.739
4	static	private	8	19.483	3.314	0.828
4	static	shared	32	19.639	3.288	0.822
4	static	private	32	19.633	3.289	0.822
4	static	shared	128	19.382	3.331	0.833
4	static	private	128	20.504	3.149	0.787
4	dynamic	shared	1	21.988	2.937	0.734
4	dynamic	private	1	19.151	3.372	0.843
4	dynamic	shared	8	17.740	3.640	0.910
4	dynamic	private	8	18.627	3.466	0.867
4	dynamic	shared	32	82.124	0.786	0.197
4	dynamic	private	32	75.445	0.856	0.214

Continua nella pagina successiva

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
4	dynamic	shared	128	182.008	0.355	0.089
4	dynamic	private	128	87.522	0.738	0.184
4	guided	shared	1	103.533	0.624	0.156
4	guided	private	1	87.076	0.742	0.185
4	guided	shared	8	96.832	0.667	0.167
4	guided	private	8	85.603	0.754	0.188
4	guided	shared	32	103.959	0.621	0.155
4	guided	private	32	84.188	0.767	0.192
4	guided	shared	128	103.538	0.624	0.156
4	guided	private	128	80.192	0.805	0.201
8	static	shared	1	20.581	3.137	0.392
8	static	private	1	52.336	1.234	0.154
8	static	shared	8	17.028	3.792	0.474
8	static	private	8	17.659	3.657	0.457
8	static	shared	32	17.326	3.727	0.466
8	static	private	32	18.716	3.450	0.431
8	static	shared	128	19.268	3.351	0.419
8	static	private	128	17.363	3.719	0.465
8	dynamic	shared	1	31.239	2.067	0.258
8	dynamic	private	1	16.441	3.928	0.491
8	dynamic	shared	8	16.037	4.026	0.503
8	dynamic	private	8	14.465	4.464	0.558
8	dynamic	shared	32	98.483	0.656	0.082
8	dynamic	private	32	78.837	0.819	0.102
8	dynamic	shared	128	97.503	0.662	0.083
8	dynamic	private	128	94.171	0.686	0.086
8	guided	shared	1	88.430	0.730	0.091
8	guided	private	1	74.602	0.866	0.108
8	guided	shared	8	94.296	0.685	0.086
8	guided	private	8	88.652	0.728	0.091
8	guided	shared	32	93.472	0.691	0.086
8	guided	private	32	74.198	0.870	0.109
8	guided	shared	128	91.159	0.708	0.089
8	guided	private	128	80.368	0.803	0.100

5.3 N = 2048 — baseline sequenziale: 2784.500 ms

Tabella 3: Dati completi per N=2048.

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
1	static	shared	1	2614.580	1.065	1.065
1	static	private	1	2270.960	1.226	1.226
1	static	shared	8	2681.080	1.039	1.039
1	static	private	8	2579.830	1.079	1.079
1	static	shared	32	2749.790	1.013	1.013
1	static	private	32	2336.270	1.192	1.192

Continua nella pagina successiva

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
1	static	shared	128	2845.870	0.978	0.978
1	static	private	128	2457.590	1.133	1.133
1	dynamic	shared	1	2813.480	0.990	0.990
1	dynamic	private	1	2562.200	1.087	1.087
1	dynamic	shared	8	2522.370	1.104	1.104
1	dynamic	private	8	2539.120	1.097	1.097
1	dynamic	shared	32	1213.180	2.295	2.295
1	dynamic	private	32	997.677	2.791	2.791
1	dynamic	shared	128	1396.680	1.994	1.994
1	dynamic	private	128	1302.170	2.138	2.138
1	guided	shared	1	2838.580	0.981	0.981
1	guided	private	1	2392.970	1.164	1.164
1	guided	shared	8	2226.200	1.251	1.251
1	guided	private	8	1912.700	1.456	1.456
1	guided	shared	32	2198.060	1.267	1.267
1	guided	private	32	1834.630	1.518	1.518
1	guided	shared	128	2872.270	0.969	0.969
1	guided	private	128	1754.000	1.588	1.588
2	static	shared	1	1289.080	2.160	1.080
2	static	private	1	1128.150	2.468	1.234
2	static	shared	8	1020.310	2.729	1.365
2	static	private	8	1226.290	2.271	1.135
2	static	shared	32	1679.500	1.658	0.829
2	static	private	32	1472.130	1.891	0.946
2	static	shared	128	1726.880	1.612	0.806
2	static	private	128	1400.460	1.988	0.994
2	dynamic	shared	1	1513.460	1.840	0.920
2	dynamic	private	1	1261.350	2.208	1.104
2	dynamic	shared	8	1186.920	2.346	1.173
2	dynamic	private	8	1303.490	2.136	1.068
2	dynamic	shared	32	593.360	4.693	2.347
2	dynamic	private	32	505.613	5.507	2.753
2	dynamic	shared	128	931.144	2.990	1.495
2	dynamic	private	128	534.790	5.207	2.603
2	guided	shared	1	2069.860	1.345	0.673
2	guided	private	1	1322.220	2.106	1.053
2	guided	shared	8	1288.340	2.161	1.081
2	guided	private	8	1194.440	2.331	1.165
2	guided	shared	32	1266.200	2.199	1.099
2	guided	private	32	1175.790	2.368	1.184
2	guided	shared	128	597.497	4.660	2.330
2	guided	private	128	476.096	5.849	2.924
4	static	shared	1	1083.960	2.569	0.642
4	static	private	1	921.931	3.020	0.755
4	static	shared	8	1018.920	2.733	0.683
4	static	private	8	908.087	3.066	0.766
4	static	shared	32	823.296	3.382	0.845

Continua nella pagina successiva

Thread	Scheduler	Variante	Chunk	Tempo [ms]	Speedup	Efficienza
4	static	private	32	882.313	3.156	0.789
4	static	shared	128	634.458	4.389	1.097
4	static	private	128	641.646	4.340	1.085
4	dynamic	shared	1	1119.030	2.488	0.622
4	dynamic	private	1	942.320	2.955	0.739
4	dynamic	shared	8	997.679	2.791	0.698
4	dynamic	private	8	913.250	3.049	0.762
4	dynamic	shared	32	342.530	8.129	2.032
4	dynamic	private	32	412.259	6.754	1.688
4	dynamic	shared	128	729.850	3.815	0.954
4	dynamic	private	128	520.061	5.354	1.338
4	guided	shared	1	677.701	4.109	1.027
4	guided	private	1	592.944	4.696	1.174
4	guided	shared	8	975.298	2.855	0.714
4	guided	private	8	492.981	5.648	1.412
4	guided	shared	32	595.617	4.675	1.169
4	guided	private	32	573.436	4.856	1.214
4	guided	shared	128	334.658	8.320	2.080
4	guided	private	128	287.783	9.676	2.419
8	static	shared	1	999.548	2.786	0.348
8	static	private	1	918.158	3.033	0.379
8	static	shared	8	988.518	2.817	0.352
8	static	private	8	905.365	3.076	0.385
8	static	shared	32	1176.100	2.368	0.296
8	static	private	32	940.740	2.960	0.370
8	static	shared	128	638.752	4.359	0.545
8	static	private	128	511.532	5.443	0.680
8	dynamic	shared	1	1015.200	2.743	0.343
8	dynamic	private	1	915.215	3.042	0.380
8	dynamic	shared	8	1006.110	2.768	0.346
8	dynamic	private	8	835.848	3.331	0.416
8	dynamic	shared	32	284.668	9.782	1.223
8	dynamic	private	32	262.123	10.623	1.328
8	dynamic	shared	128	493.164	5.646	0.706
8	dynamic	private	128	911.009	3.057	0.382
8	guided	shared	1	430.772	6.464	0.808
8	guided	private	1	739.024	3.768	0.471
8	guided	shared	8	541.928	5.138	0.642
8	guided	private	8	716.066	3.889	0.486
8	guided	shared	32	553.072	5.035	0.629
8	guided	private	32	809.406	3.440	0.430
8	guided	shared	128	284.877	9.774	1.222
8	guided	private	128	259.678	10.723	1.340

6 Studio delle variabili condivise/private

6.1 Differenza architetturale

Nella variante **shared**, ogni thread accumula direttamente sulla riga i di C . Non vi sono race condition (thread diversi scrivono su righe diverse), ma altri thread potrebbero accedere a righe adiacenti che cadono nella stessa cache line (64 byte = 16 float), causando **false sharing**.

Nella variante **private**, ogni thread accumula su un buffer locale `row_buf` sullo stack, eliminando il false sharing. La scrittura su C avviene una sola volta per riga.

6.2 Risultati e discussione

Tabella 4: Speedup shared vs private (scheduler: dynamic, miglior chunk size).

	N=512		N=1024		N=2048	
Thread	shared	private	shared	private	shared	private
1	3.966	3.566	1.016	0.858	2.295	2.791
2	7.708	5.324	1.876	1.810	4.693	5.507
4	10.777	12.637	3.640	3.466	8.129	6.754
8	10.378	12.889	4.026	4.464	9.782	10.623

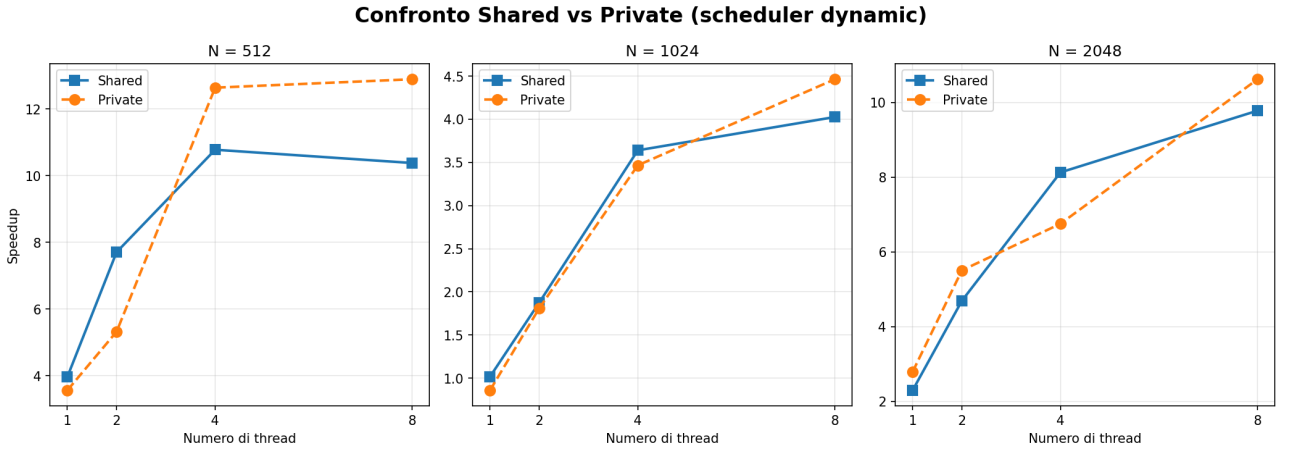


Figura 1: Speedup shared vs private (dynamic, miglior chunk) al variare del numero di thread.

N=512: con 1–2 thread la shared è superiore (il buffer privato introduce overhead di inizializzazione non compensato). Con 4–8 thread si inverte: private raggiunge $12.9\times$ contro $10.4\times$ della shared, effetto del false sharing che si accentua con molti thread che scrivono su righe adiacenti di C .

N=1024: differenze contenute fino a 4 thread ($< 5\%$). A 8 thread la private supera di $+10.9\%$ ($4.464\times$ vs $4.026\times$).

N=2048: comportamento non monotono; con 8 thread la private è superiore di $+8.6\%$ ($10.623\times$ vs $9.782\times$). La variante private è raccomandata con molti thread per qualsiasi dimensione.

7 Studio dello scheduling dei thread

7.1 I tre scheduler

- **Static:** blocchi di dimensione *chunk* assegnati ciclicamente prima dell'esecuzione. Overhead minimo, nessuna adattabilità.
- **Dynamic:** chunk assegnati a richiesta durante l'esecuzione. Maggiore overhead, bilanciamento ottimale.
- **Guided:** chunk size decrescente. Il parametro *chunk* specifica la dimensione minima.

7.2 Confronto scheduler e chunk size

Tabella 5: Miglior speedup per scheduler (8 thread, variante private).

N	Scheduler	Chunk ottimale	Tempo [ms]	Speedup
512	static	128	2.630	10.348
	dynamic	1	2.112	12.889
	guided	32	1.567	17.369
1024	static	128	17.363	3.719
	dynamic	8	14.465	4.464
	guided	32	74.198	0.870
2048	static	128	511.532	5.443
	dynamic	32	262.123	10.623
	guided	128	259.678	10.723

Tabella 6: Speedup al variare del chunk size (8 thread, scheduler static).

Chunk	N=512		N=1024		N=2048	
	shared	private	shared	private	shared	private
1	4.295	2.293	3.137	1.234	2.786	3.033
8	5.384	2.353	3.792	3.657	2.817	3.076
32	1.775	1.763	3.727	3.450	2.368	2.960
128	2.756	10.348	3.351	3.719	4.359	5.443

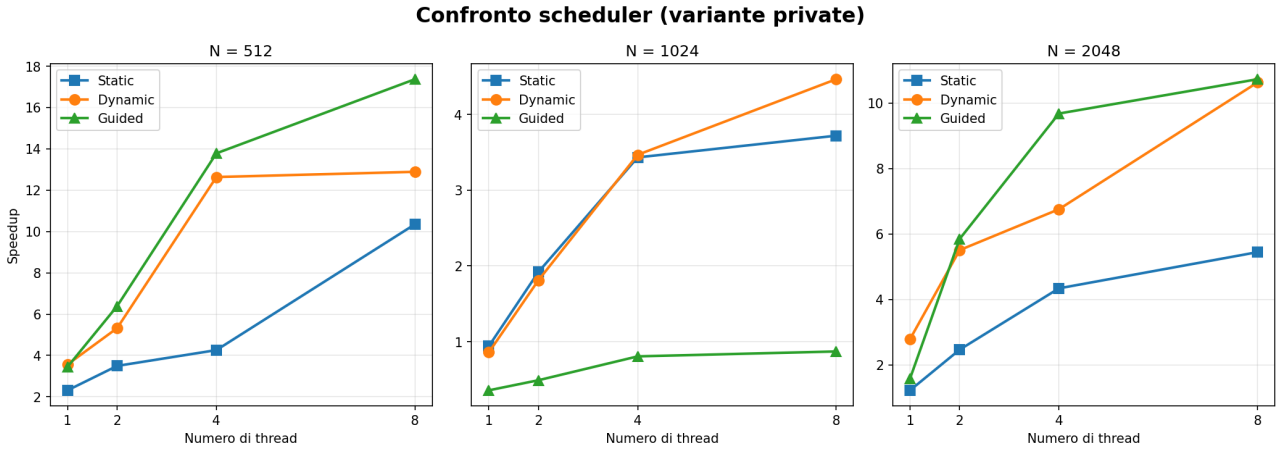


Figura 2: Confronto scheduler (8 thread, variante privata, miglior chunk).

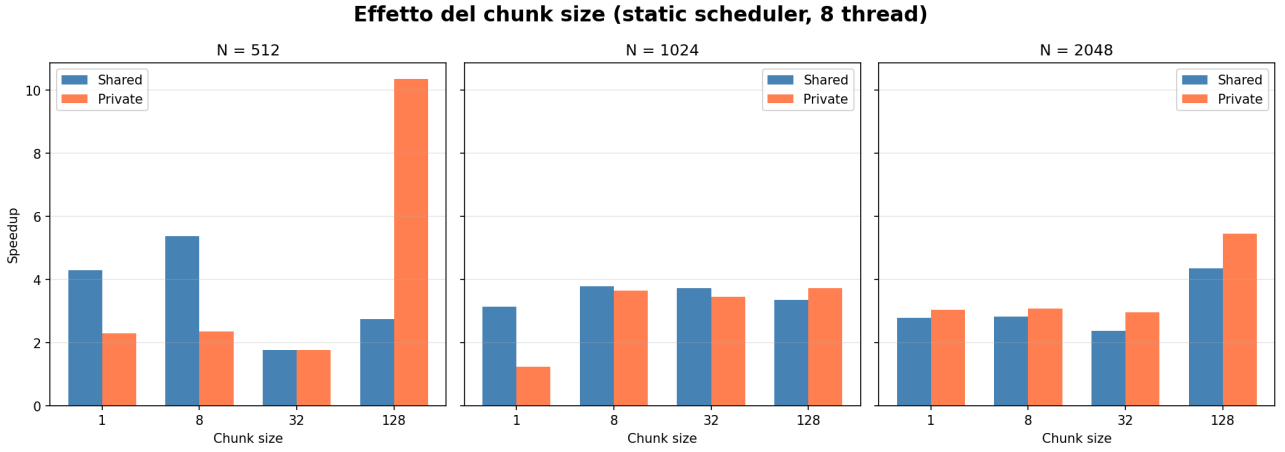


Figura 3: Effetto del chunk size (8 thread, static, shared e private).

N=512: guided è il migliore ($17.4\times$).

N=1024: guided produce speedup < 1 per tutte le configurazioni — l'overhead di ricalcolo del chunk supera il beneficio; dynamic è il migliore ($4.46\times$).

N=2048: guided e dynamic sono equivalenti ($10.72\times$ e $10.62\times$), entrambi nettamente superiori a static ($5.44\times$). Il chunk size ha impatto critico: per static con $N=512$, private chunk=128 dà $10.3\times$ contro $1.8\times$ di private chunk=32 — un fattore $6\times$ a parità di tutti gli altri parametri.

8 Studio delle performance al variare della dimensione e del parallelismo

Tabella 7: Configurazione ottimale per dimensione.

N	T_{seq} [ms]	Thread	Scheduler	Chunk	Variante	T_{par} [ms]	Speedup
512	27.22	8	guided	32	private	1.567	17.37$\times$
1024	64.57	8	dynamic	8	private	14.465	4.46$\times$
2048	2784.50	8	guided	128	private	259.678	10.72$\times$

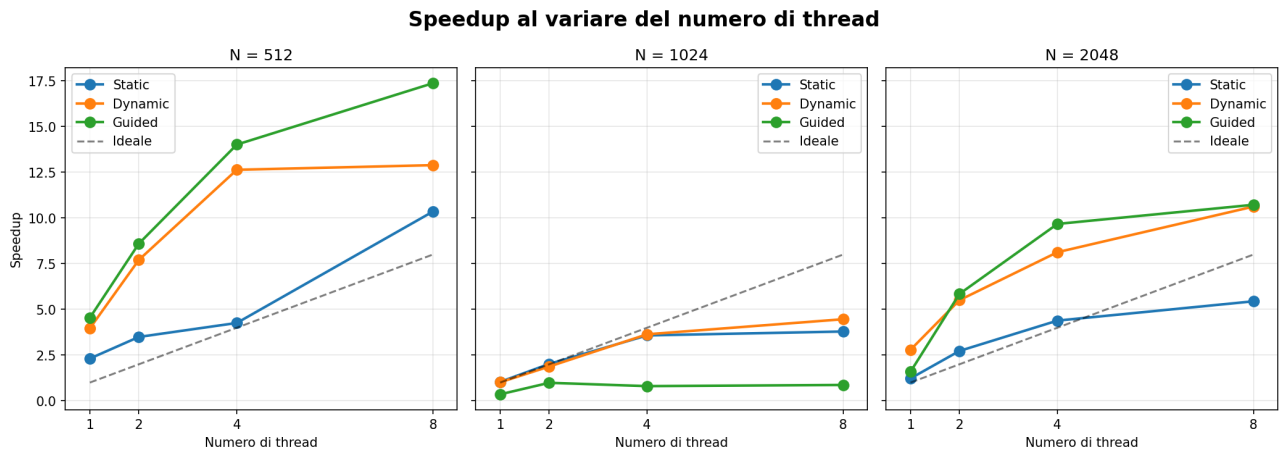


Figura 4: Speedup assoluto al variare del numero di thread e dello scheduler.

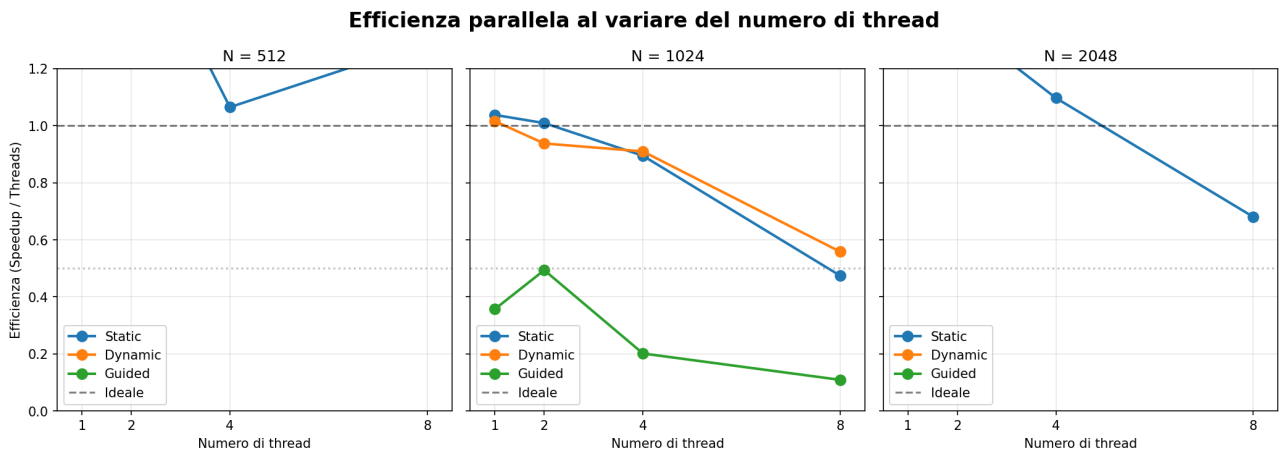


Figura 5: Efficienza parallela $E = S/N_{thread}$ al variare del numero di thread.

Speedup superlineare per N=512: lo speedup $17.4\times$ supera il numero di thread fisici perché ogni thread beneficia di cache L1 private, mentre la versione sequenziale soffre di più evictions sull'intera matrice.

N=1024: speedup massimo più basso ($4.5\times$) nonostante non sia la dimensione più grande. Con N=2048 il tempo per iterazione è molto più lungo e nasconde meglio le latenze di memoria, producendo speedup più elevati.

Distribuzione del lavoro: la variante per righe è superiore alla per elemento con `collapse(2)`, perché preserva la località di accesso dell'ordinamento `ikj`.

9 Conclusioni

- **Dimensione:** speedup superlineare per N=512; overhead dominante di guided per N=1024; speedup elevato per N=2048.
- **Granularità:** parallelismo per righe superiore al per elemento con `collapse(2)`.
- **Shared vs private:** private preferibile con molti thread ($\sim +10\%$ a 8 thread).
- **Scheduler:** guided ottimale per N=512; dynamic per N=1024; equivalenti per N=2048.

- **Chunk size:** impatto fino a $6\times$ a parità degli altri parametri.

Tabella 8: Configurazioni raccomandate.

Dimensione	Thread	Scheduler	Chunk	Variante	Speedup atteso
N=512	8	guided	32	private	$\sim 17\times$
N=1024	8	dynamic	8	private	$\sim 4.5\times$
N=2048	8	guided	128	private	$\sim 11\times$